

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №2
по дисциплине «Машинное обучение»
Тема: Автоматизированный выбор и настройка моделей машинного
обучения с использованием Auto-sklearn.

Студент гр. 1306

Шаврин А.П.

Преподаватель

Куликов И.А.

Санкт-Петербург

2025

Цель работы.

Изучить принципы автоматизированного машинного обучения (AutoML) и освоить практические навыки применения библиотеки Auto-sklearn для автоматического выбора алгоритмов машинного обучения, настройки их гиперпараметров и формирования ансамблей моделей при решении задач классификации и регрессии.

Основные теоретические положения.

Современные задачи машинного обучения требуют подбора оптимального алгоритма и его гиперпараметров, что является трудоёмким и вычислительно затратным процессом. Подход **AutoML (Automated Machine Learning)** направлен на автоматизацию следующих этапов:

- предварительная обработка данных;
- выбор типа модели;
- настройка гиперпараметров;
- оценка качества моделей;
- формирование ансамбля лучших решений.

Auto-sklearn — это библиотека автоматизированного машинного обучения на языке Python, построенная поверх библиотеки Scikit-learn. Она использует методы байесовской оптимизации для подбора гиперпараметров, а также включает два дополнительных механизма повышения качества моделей:

- мета-обучение, основанное на анализе мета-свойств ранее изученных наборов данных из репозитория OpenML;
- оркестровку (ансамблирование), заключающуюся в объединении нескольких лучших моделей в итоговое решение.

Auto-sklearn автоматически управляет вычислительным бюджетом, последовательно обучая различные конфигурации моделей и формируя лидерборд — список лучших решений, отсортированных по выбранной метрике качества.

Постановка задачи.

В рамках лабораторной работы требуется исследовать возможности автоматизированного машинного обучения с использованием Auto-sklearn при решении двух практических задач.

Задача 1. Классификация

Исследовать набор данных №42721 из репозитория OpenML, содержащий информацию о задержках авиарейсов. Требуется спрогнозировать величину задержки рейса. Необходимо:

- загрузить набор данных;
- вывести описание и основную информацию о данных;
- разбить данные на обучающую и тестовую выборки;
- с помощью Auto-sklearn автоматически подобрать регрессионные модели;
- оценить качество прогнозирования на тестовых данных;
- увеличить количество используемых вычислительных потоков и сравнить результаты.

Задача 2. Регрессия

Исследовать набор данных `smoke_detection.csv`, содержащий показания датчиков для определения факта возгорания. Необходимо:

- загрузить набор данных из файла;
- выполнить разбиение на обучающую и тестовую выборки;
- использовать Auto-sklearn для подбора классификаторов;
- вывести информацию о проделанном поиске и найденных моделях;
- оценить точность классификации на тестовых данных;
- уменьшить время поиска моделей с сохранением качества.

Выполнение работы.

1. Прогнозирование задержек авиарейсов (OpenML dataset №42721)

Для выполнения первой части лабораторной работы была реализована функция `run_airdelays_task`. Данная функция объединяет все основные этапы работы с набором данных: загрузку, анализ, разбиение, обучение моделей Auto-sklearn и оценку результатов.

Загрузка данных выполняется с использованием репозитория OpenML:

```
dataset = openml.datasets.get_dataset(dataset_id)
```

```
X, y, _, _ = dataset.get_data(target=dataset.default_target_attribute)
```

После загрузки выводится размерность набора данных, его описание и базовая информация о признаках, что позволяет получить общее представление о структуре и типах данных (рисунок 1).

```
(1000000, 9)
=== DESCRIPTION ===
Описание набора данных:
**Author**: Bureau of Transportation Statistics, Airline Service Quality Performance
**Source**: [original](http://www.transtats.bts.gov/) - 2013
**Please cite**:

Airlines Departure Delay Prediction (Regression).
Original data can be found at: http://www.transtats.bts.gov

This is a processed version of the original data, designed to predict departure delay (in seconds).

A CSV of the raw data (years 1987-2013) can be found [here](https://h2o-airlines-unpacked.s3.amazonaws.com/allyears.1987.2013.csv). This is the first 1 million rows (and a subset of the columns) of this CSV file, in ARFF format.
=== INFO ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Month                 1000000 non-null  uint8
1   DayOfMonth            1000000 non-null  uint8
2   DayOfWeek             1000000 non-null  uint8
3   CRSDepTime            1000000 non-null  float64
4   CRSArrTime            1000000 non-null  float64
5   UniqueCarrier         1000000 non-null  category
6   Origin                1000000 non-null  category
7   Dest                 1000000 non-null  category
8   Distance              1000000 non-null  float64
dtypes: category(3), float64(3), uint8(3)
memory usage: 30.5 MB
None
```

Рисунок 1. Описание датасета

Далее выполняется разбиение данных на обучающую и тестовую выборки в соотношении 80% к 20% с использованием функции `train_test_split` из библиотеки `sklearn`. Фиксированное значение параметра `random_state` обеспечивает воспроизводимость результатов.

Для автоматического подбора регрессионных моделей используется класс `AutoSklearnRegressor`. При его инициализации задаётся ограничение по времени поиска (`time_left_for_this_task`) и количество используемых потоков (`n_jobs`), что позволяет управлять вычислительными ресурсами.

```
regressor = autosklearn.regression.AutoSklearnRegressor(
    time_left_for_this_task=time_limit,
    n_jobs=n_jobs
)
```

На рисунке 2 представлен скриншот запущенного процесса обучения с ограничением в 500с в одном процессе.

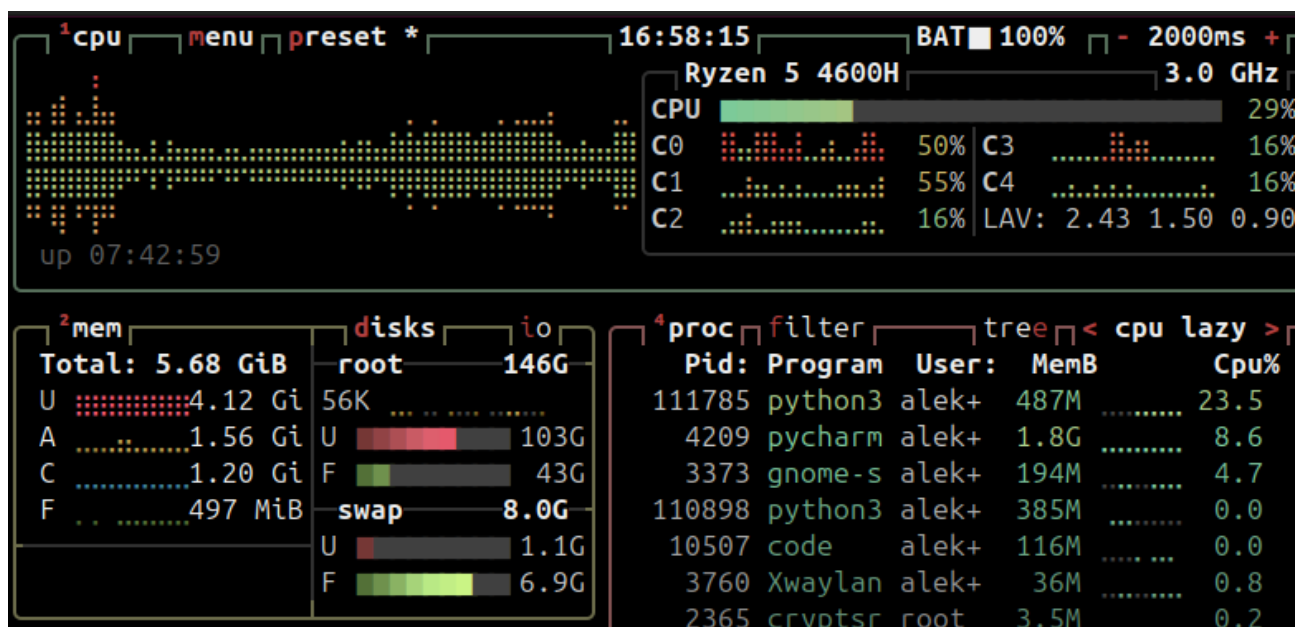


Рисунок 2. Запущенная программа в одном процессе.

После обучения модели выполняется прогнозирование на тестовой выборке и вычисление основных метрик качества регрессии (MAE, RMSE, R2). Результаты оценки модели представлены на рисунке 3.

```
=== METRICS ===
MAE: 11.84115400937429
RMSE: 26.957541454912995
R^2: 0.030600844942997862
```

Рисунок 3. Оценка точности на тестовых данных.

При ограничении времени поиска в 500 секунд и использовании одного вычислительного процесса качество построенной регрессионной модели остаётся низким. Значение коэффициента детерминации R^2 близко к нулю, что указывает на крайне слабую объясняющую способность модели и фактически означает, что полученный прогноз лишь незначительно превосходит наивную модель, основанную на среднем значении целевой переменной. Значения MAE и RMSE остаются высокими, что свидетельствует о значительных абсолютных и квадратичных ошибках прогнозирования. Это указывает на то, что найденные модели не способны адекватно описать сложные зависимости в данных при заданных вычислительных ограничениях.

Также выводится информация о проделанном поиске, лидерборд найденных моделей и подробное описание лучших конфигураций с помощью методов `sprint_statistics`, `leaderboard` и `show_models` (рисунок 4).

```

=== SEARCH STATISTICS ===
auto-sklearn results:
  Dataset name: 6e5597be-db6c-11f0-bd3b-cd69d84c1163
  Metric: r2
  Best validation score: 0.035291
  Number of target algorithm runs: 29
  Number of successful target algorithm runs: 5
  Number of crashed target algorithm runs: 6
  Number of target algorithms that exceeded the time limit: 7
  Number of target algorithms that exceeded the memory limit: 11

=== LEADERBOARD ===
      rank  ensemble_weight      type      cost  duration
model_id
11         1           0.84  gradient_boosting  0.964709  41.603880
25         2           0.12  gradient_boosting  0.994757   7.038636
9          3           0.04    ard_regression  0.996872   3.859666

=== MODELS INFO ===
      model_id  rank      cost  ensemble_weight      sklearn_regressor
9           9     1  0.996872           0.04  ARDRegression(alpha_1=4.7044575285722365e-05, ...
11          11     2  0.964709           0.84  HistGradientBoostingRegressor(l2_regularizatio...
25          25     3  0.994757           0.12  HistGradientBoostingRegressor(l2_regularizatio...

```

Рисунок 4. Статистика.

Анализ статистики Auto-sklearn показывает, что при использовании одного потока значительная часть запусков алгоритмов завершилась неуспешно. Существенное количество конфигураций превысило лимиты по времени и памяти, а также наблюдались аварийные завершения отдельных запусков. При

этом общее число успешных запусков оказалось небольшим по сравнению с общим числом попыток. В итоговый ансамбль вошло ограниченное число моделей, при доминирующей роли градиентного бустинга, что указывает на сужение пространства эффективного поиска. Несмотря на относительно большое количество выполненных запусков, качество лучшей валидационной конфигурации осталось низким, что говорит о недостаточной эффективности поиска в заданных условиях.

Затем было увеличено число процессов до 4х и проведено обучение повторно (рисунки 5 - 7).

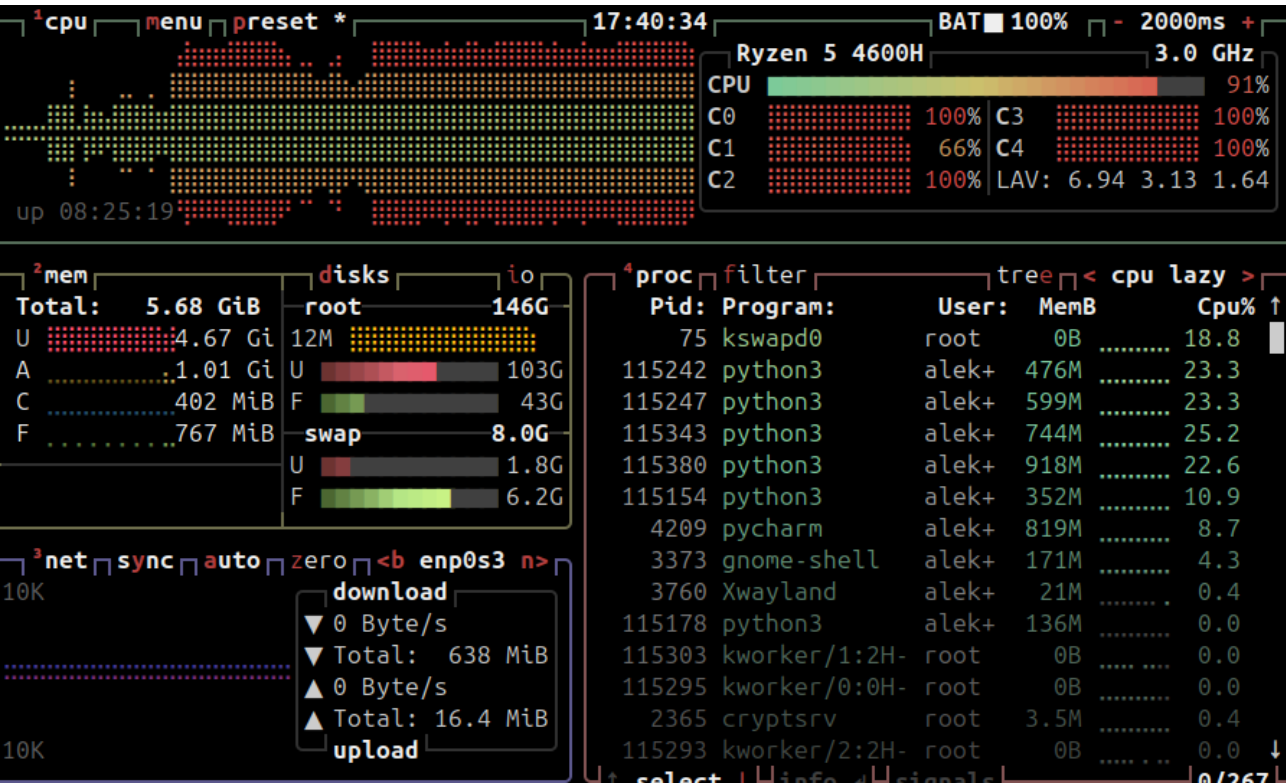


Рисунок 5. Запущенная программа в 4х процессах.

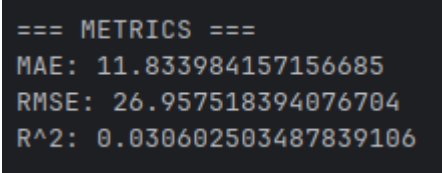


Рисунок 6. Оценка точности на тестовых данных.

```

=== SEARCH STATISTICS ===
auto-sklearn results:
Dataset name: 18da202b-db6f-11f0-81d2-7d0741b91d08
Metric: r2
Best validation score: 0.035291
Number of target algorithm runs: 23
Number of successful target algorithm runs: 3
Number of crashed target algorithm runs: 6
Number of target algorithms that exceeded the time limit: 9
Number of target algorithms that exceeded the memory limit: 5

=== LEADERBOARD ===

```

model_id	rank	ensemble_weight	type	cost	duration
11	1	0.86	gradient_boosting	0.964709	77.277748
9	2	0.14	ard_regression	0.996872	10.449825

```

=== MODELS INFO ===

```

model_id	rank	cost	ensemble_weight	sklearn_regressor
9	9	1	0.996872	0.14 ARDRegression(alpha_1=4.7044575285722365e-05, ...)
11	11	2	0.964709	0.86 HistGradientBoostingRegressor(l2_regularizatio...

Рисунок 7. Статистика.

Увеличение числа используемых процессов с одного до четырёх при том же временном ограничении не привело к заметному улучшению качества модели. Значения MAE, RMSE и коэффициента детерминации R^2 остались практически неизменными, что свидетельствует об отсутствии выигрыша в точности прогнозирования. Это указывает на то, что увеличение параллелизма в данном случае не позволило Auto-sklearn найти принципиально более качественные конфигурации моделей.

При этом статистика поиска показывает перераспределение вычислительных ресурсов: уменьшилось общее число запусков алгоритмов, однако доля конфигураций, превысивших временные ограничения, увеличилась. Это объясняется тем, что при параллельном выполнении несколько ресурсоёмких моделей одновременно конкурируют за вычислительные ресурсы, что приводит к более частому превышению лимитов времени. Итоговый ансамбль также сократился по составу и стал более концентрированным вокруг одной доминирующей модели.

В целом увеличение числа процессов привело главным образом к изменению характера поиска, но не к улучшению качества итоговой модели. Полученные результаты показывают, что в условиях жёстких ограничений по

времени и высокой сложности данных распараллеливание само по себе не гарантирует повышения точности и может сопровождаться ростом числа неуспешных запусков.

2. Обнаружение возгорания по данным датчиков (smoke_detection.csv)

Для решения задачи классификации была реализована функция `run_smoke_detection_task`, включающая этапы загрузки данных, анализа, обучения классификаторов и оценки качества.

Загрузка данных осуществляется из CSV-файла с использованием библиотеки `pandas`. После загрузки выводится информация о структуре набора данных, первые строки и описательная статистика признаков (рисунок 8).

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 62630 entries, 0 to 62629
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            62630 non-null  int64
1   UTC                   62630 non-null  int64
2   Temperature[C]       62630 non-null  float64
3   Humidity[%]          62630 non-null  float64
4   TVOC[ppb]            62630 non-null  int64
5   eCO2[ppm]            62630 non-null  int64
6   Raw H2                62630 non-null  int64
7   Raw Ethanol          62630 non-null  int64
8   Pressure[hPa]        62630 non-null  float64
9   PM1.0                62630 non-null  float64
10  PM2.5                62630 non-null  float64
11  NC0.5                62630 non-null  float64
12  NC1.0                62630 non-null  float64
13  NC2.5                62630 non-null  float64
14  CNT                  62630 non-null  int64
15  Fire Alarm           62630 non-null  int64
dtypes: float64(8), int64(8)
memory usage: 7.6 MB
None
```

Рисунок 8. Описание датасета

Целевая переменная отделяется от признаков, после чего данные разбиваются на обучающую и тестовую выборки в соотношении 80% к 20%.

Для автоматического подбора классификаторов используется класс `AutoSklearnClassifier`. В параметрах задаются общее время поиска, лимит времени на одну конфигурацию и количество используемых потоков:

```
cls = autosklearn.classification.AutoSklearnClassifier(  
    time_left_for_this_task=time_limit,  
    per_run_time_limit=time_limit // 10,  
    n_jobs=n_jobs  
)
```

После обучения выполняется предсказание на тестовой выборке и вычисление метрик точности классификации (рисунок 9) Дополнительно проводится проверка отсутствия утечки данных между обучающей и тестовой выборками.

```
=== METRICS ===  
Overlap: 0  
Target in X: False  
Accuracy: 1.0  
  
Classification report:  
              precision    recall  f1-score   support  
0               1.00      1.00      1.00     3594.0  
1               1.00      1.00      1.00     8932.0  
accuracy               1.00      1.00      1.00         1.0  
macro avg              1.00      1.00      1.00    12526.0  
weighted avg           1.00      1.00      1.00    12526.0
```

Рисунок 9. Оценка точности на тестовых данных.

По результатам оценки на тестовой выборке модель продемонстрировала идеальные показатели качества классификации. Значение Accuracy равно 1.0, а показатели precision, recall и F1-score для обоих классов также равны 1.0. Это означает, что все объекты тестовой выборки были классифицированы корректно без единой ошибки. Дополнительные проверки подтверждают корректность эксперимента: пересечений между обучающей и тестовой выборками не обнаружено (Overlap = 0), а целевая переменная отсутствует среди признаков, что исключает утечку данных. Таким образом, полученные метрики отражают реальное качество модели на новых данных.

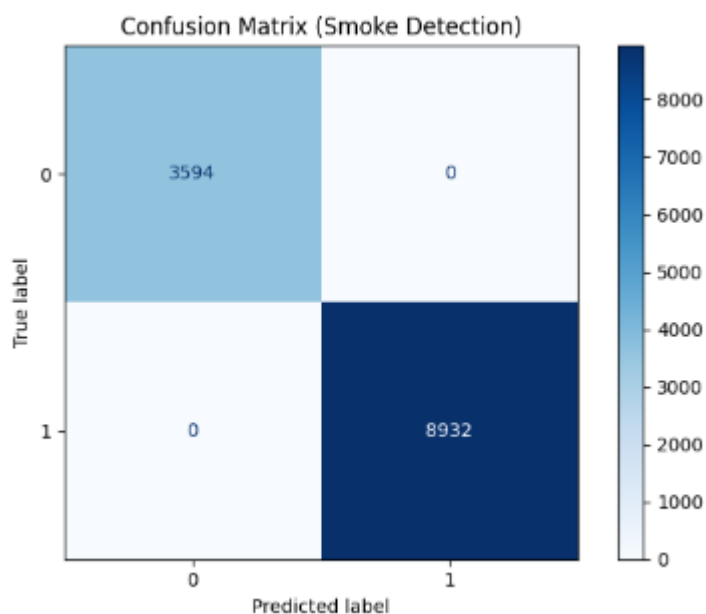


Рисунок 10. ConfusionMatrix.

Визуализация матрицы ошибок (рисунок 10) также демонстрирует полное предсказывание меток классов.

```
=== SEARCH STATISTICS ===
auto-sklearn results:
Dataset name: f6bd1909-db75-11f0-8ecd-7b2ae0a8229a
Metric: accuracy
Best validation score: 0.999819
Number of target algorithm runs: 27
Number of successful target algorithm runs: 8
Number of crashed target algorithm runs: 13
Number of target algorithms that exceeded the time limit: 6
Number of target algorithms that exceeded the memory limit: 0
```

Рисунок 11. Статистика.

Статистика Auto-sklearn показывает, что в ходе автоматического поиска было запущено 27 конфигураций моделей, из которых 8 завершились успешно. Часть запусков завершилась с ошибками или превысила лимиты времени, что является типичной ситуацией при работе AutoML в условиях ограниченного вычислительного бюджета и большого пространства поиска. При этом наилучшее значение метрики на валидации составило 0.999819, что практически совпадает с результатами на тестовой выборке. Это указывает на то, что Auto-sklearn сумел найти устойчивые и хорошо обобщающиеся решения, несмотря на ограниченное число успешных запусков.

```

=== LEADERBOARD ===

```

	rank	ensemble_weight	type	cost	duration
model_id					
2	1	0.06	random_forest	0.000181	23.424378
19	2	0.38	mlp	0.000181	24.335079
5	3	0.02	gradient_boosting	0.000242	9.336845
22	4	0.02	gradient_boosting	0.000302	6.658301
20	5	0.20	extra_trees	0.000423	11.502687
10	6	0.26	random_forest	0.000484	25.498332
23	7	0.04	sgd	0.004596	3.767049
15	8	0.02	mlp	0.006713	39.966355

Рисунок 12. Лидерборд.

Лидерборд показывает, что в итоговый ансамбль вошли модели различных типов, включая Random Forest, MLP, Gradient Boosting, Extra Trees и SGD. Лучшие конфигурации имеют крайне малую величину ошибки (cost близок к нулю), что соответствует высокой точности классификации. При этом ни одна модель не доминирует полностью, а итоговое решение формируется за счёт ансамблирования нескольких сильных алгоритмов. Это подтверждает эффективность оркестровки, используемой в Auto-sklearn, при которой объединение разнородных моделей обеспечивает более стабильное и надёжное качество, чем использование одного классификатора.

```

=== MODELS INFO ===
{0: {'model_id': 2, 'rank': 1, 'cost': 0.00018143322537102, 'ensemble_weight': 0.06, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x7dc046ac8cd0>, 'balancing': Balancing(random_state=1), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x7dc046a13190>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x7dc046a13200>, 'sklearn_classifier': RandomForestClassifier(max_features=3, n_estimators=512, n_jobs=1, random_state=1, warm_start=True)}, 5: {'model_id': 5, 'rank': 2, 'cost': 0.0002419110976715766, 'ensemble_weight': 0.02, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x7dc046a2b8e0>, 'balancing': Balancing(random_state=1), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x7dc046afff10>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x7dc046aff910>, 'sklearn_classifier': HistGradientBoostingClassifier(early_stopping=True, learning_rate=0.010827728124541558, max_iter=512, max_leaf_nodes=25, min_samples_leaf=4, n_iter_no_change=19, random_state=1, validation_fraction=0.1759114608225633, warm_start=True)}, 10: {'model_id': 10, 'rank': 3, 'cost': 0.0004838221953432642, 'ensemble_weight': 0.26, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x7dc046a2b8e0>, 'balancing': Balancing(random_state=1), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x7dc046a460d0>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x7dc046a462b0>, 'sklearn_classifier': RandomForestClassifier(max_features=3, min_samples_leaf=2, min_samples_split=20, n_estimators=512, n_jobs=1, random_state=1, warm_start=True)}, 15: {'model_id': 15, 'rank': 4, 'cost': 0.006713032960387055, 'ensemble_weight': 0.02, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x7dc046a2b8e0>, 'balancing': Balancing(random_state=1), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x7dc046a470940>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x7dc046a478ac0>, 'sklearn_classifier': MLPClassifier(activation='tanh', alpha=3.542372581684717e-07, beta_1=0.999, beta_2=0.9, hidden_layer_sizes=(54, 54), learning_rate_init=0.00016753718126302934, max_iter=128, n_iter_no_change=32, random_state=1, validation_fraction=0.0, verbose=0, warm_start=True)}, 19: {'model_id': 19, 'rank': 5, 'cost': 0.00018143322537102, 'ensemble_weight': 0.38, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x7dc046a2b8e0>, 'balancing': Balancing(random_state=1, strategy='weighting'), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x7dc046a4679d0>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x7dc046a46cdd0>, 'sklearn_classifier': MLPClassifier(activation='tanh', alpha=0.0288316953521873, beta_1=0.999, beta_2=0.9, hidden_layer_sizes=(91,), learning_rate_init=0.0002389951929622804, max_iter=64, n_iter_no_change=32, random_state=1, validation_fraction=0.0, verbose=0, warm_start=True)}, 20: {'model_id': 20, 'rank': 6, 'cost': 0.0004233444209252868, 'ensemble_weight': 0.2, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x7dc046a4679d0>, 'balancing': Balancing(random_state=1), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x7dc046a4679d0>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x7dc046a4679d0>, 'sklearn_classifier': ExtraTreesClassifier(criterion='entropy', max_features=4, min_samples_leaf=2, min_samples_split=15, n_estimators=512, n_jobs=1, random_state=1, warm_start=True)}, 22: {'model_id': 22, 'rank': 7, 'cost': 0.00030238872089554, 'ensemble_weight': 0.02, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x7dc046a4679d0>, 'balancing': Balancing(random_state=1, strategy='weighting'), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x7dc046a4679d0>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x7dc046a4679d0>, 'sklearn_classifier': HistGradientBoostingClassifier(early_stopping=False, learning_rate=0.11042628136263043, max_iter=512, max_leaf_nodes=30, min_samples_leaf=22, n_iter_no_change=0, random_state=1, validation_fraction=None, warm_start=True)}, 23: {'model_id': 23, 'rank': 8, 'cost': 0.00459631085796051, 'ensemble_weight': 0.04, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x7dc046a4679d0>, 'balancing': Balancing(random_state=1), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x7dc046a4679d0>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x7dc046a4679d0>, 'sklearn_classifier': SGDClassifier(alpha=0.0002346515712097644, average=True, eta0=0.01, loss

```

Рисунок 13. Веса моделей.

Анализ весов моделей в ансамбле показывает, что наибольший вклад в итоговое решение вносят нейронные сети MLP и модели на основе деревьев решений. Наиболее значимыми являются MLPClassifier с весом 0.38 и

RandomForestClassifier с весом 0.26, а также ExtraTreesClassifier с весом 0.20. Остальные модели имеют меньшие веса и выполняют вспомогательную роль, повышая устойчивость ансамбля. Таким образом, итоговая модель опирается в основном на несколько сильных нелинейных алгоритмов, которые эффективно выявляют сложные зависимости в данных, а остальные классификаторы используются для сглаживания ошибок и повышения надёжности предсказаний.

Затем время выполнения было снижено с 500с до 300с, результаты обучения представлены на рисунках 14-18.

```

=== METRICS ===
Overlap: 0
Target in X: False
Accuracy: 1.0

Classification report:
              precision    recall  f1-score   support
0               1.0         1.0         1.0     3594.0
1               1.0         1.0         1.0     8932.0
accuracy               1.0         1.0         1.0         1.0
macro avg              1.0         1.0         1.0    12526.0
weighted avg            1.0         1.0         1.0    12526.0

```

Рисунок 14. Оценка точности на тестовых данных.

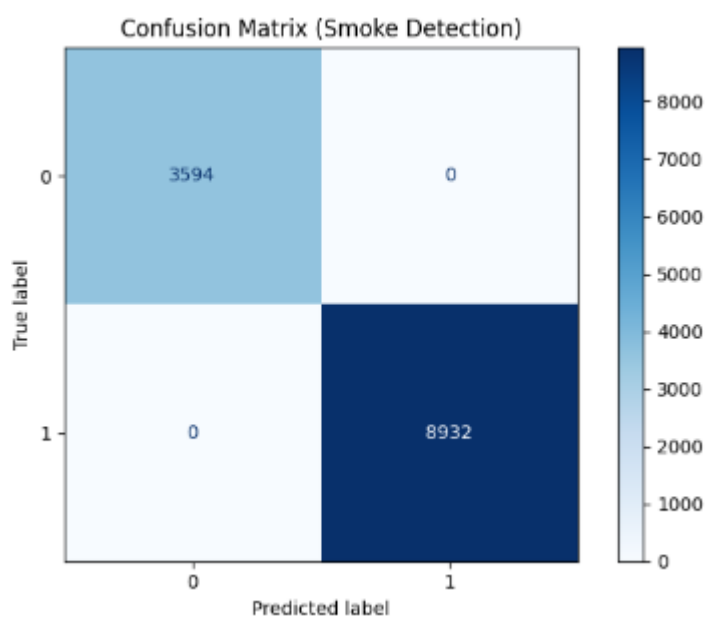


Рисунок 15. ConfusionMatrix.

```

=== SEARCH STATISTICS ===
auto-sklearn results:
Dataset name: 0716713e-db7d-11f0-9ede-fb10d0a436f5
Metric: accuracy
Best validation score: 0.999819
Number of target algorithm runs: 25
Number of successful target algorithm runs: 7
Number of crashed target algorithm runs: 12
Number of target algorithms that exceeded the time limit: 6
Number of target algorithms that exceeded the memory limit: 0

```

Рисунок 16. Статистика.

```

=== LEADERBOARD ===

```

	rank	ensemble_weight	type	cost	duration
model_id					
2	1	0.04	random_forest	0.000181	22.539228
19	2	0.34	mlp	0.000181	19.755358
22	3	0.04	gradient_boosting	0.000302	5.657195
20	4	0.28	extra_trees	0.000423	9.191863
10	5	0.26	random_forest	0.000484	24.700519
23	6	0.04	sgd	0.004596	3.076428

Рисунок 17. Лидерборд

```

=== MODELS INFO ===
(2: {'model_id': 2, 'rank': 1, 'cost': 0.0001814333232537102, 'ensemble_weight': 0.04, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x780b37cd2c18>, 'balancing': Balancing(random_state=1), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x780b37c0e490>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x780b37c0e3a0>, 'sklearn_classifier': RandomForestClassifier(max_features=3, n_estimators=512, n_jobs=1, random_state=1, warm_start=True)}, 10: {'model_id': 10, 'rank': 5, 'cost': 0.00048438221953432642, 'ensemble_weight': 0.26, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x780b37c0e490>, 'balancing': Balancing(random_state=1), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x780b37c42550>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x780b37c4e800>, 'sklearn_classifier': MLPClassifier(activation='tanh', alpha=0.0288316953521873, beta_1=0.999, n_estimators=512, n_jobs=1, random_state=1, warm_start=True)}, 19: {'model_id': 19, 'rank': 2, 'cost': 0.0001814333232537102, 'ensemble_weight': 0.34, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x780b37c0e3a0>, 'balancing': Balancing(random_state=1, strategy='weighting'), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x780b37c0e490>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x780b37c0e800>, 'sklearn_classifier': ExtraTreesClassifier(criterion='entropy', max_features=4, min_samples_leaf=2, min_samples_split=15, n_estimators=512, n_jobs=1, random_state=1, warm_start=True)}, 20: {'model_id': 20, 'rank': 4, 'cost': 0.000423444209252868, 'ensemble_weight': 0.28, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x780b37c0e3a0>, 'balancing': Balancing(random_state=1), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x780b37c0e490>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x780b37c0e800>, 'sklearn_classifier': HistGradientBoostingClassifier(early_stopping=False, learning_rate=0.1042628136263043, max_iter=512, max_leaf_nodes=30, min_samples_leaf=22, n_iter_no_change=0, random_state=1, validation_fraction=0.0, verbose=0, warm_start=True)}, 22: {'model_id': 22, 'rank': 3, 'cost': 0.00030238872889554, 'ensemble_weight': 0.04, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x780b37c0e3a0>, 'balancing': Balancing(random_state=1, strategy='weighting'), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x780b37c0e490>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x780b37c0e800>, 'sklearn_classifier': SGDClassifier(alpha=0.0002346515712987664, average=True, eta0=0.01, loss='log', max_iter=256, penalty='l1', random_state=1, tol=1.371648938647322e-05, warm_start=True)}).

```

Рисунок 18. Веса моделей.

Сравнение результатов работы Auto-sklearn при ограничении времени 300 и 500 секунд показывает, что увеличение вычислительного бюджета не привело к улучшению качества классификации на тестовой выборке. В обоих экспериментах итоговая ансамблевая модель продемонстрировала идеальную точность, равную 1.0, при этом значения precision, recall и F1-score для всех классов также составили 1.0. Матрица ошибок не содержит неверно классифицированных объектов, что указывает на полное совпадение

предсказаний модели с истинными метками тестовых данных уже при меньшем временном ограничении.

Анализ статистики поиска показывает, что при увеличении лимита времени Auto-sklearn выполнил немного больше запусков целевых алгоритмов и нашёл на одну успешную конфигурацию больше. Однако наилучшее значение метрики качества на валидационных данных осталось неизменным, что свидетельствует о достижении плато качества. Дополнительное время в основном было затрачено на исследование более сложных или альтернативных конфигураций моделей, часть из которых завершалась с ошибками или превышением временных ограничений, не улучшая итоговый результат.

Лидерборды для обоих экспериментов имеют схожую структуру и включают одинаковые типы моделей, такие как Random Forest, MLP, Extra Trees, Gradient Boosting и SGD. При увеличении времени поиска ансамбль стал более насыщенным и диверсифицированным, а веса отдельных моделей перераспределились более равномерно. Наибольший вклад по-прежнему вносят модели Random Forest и MLP, в то время как остальные алгоритмы играют вспомогательную роль и повышают устойчивость ансамбля.

Таким образом, увеличение времени работы Auto-sklearn с 300 до 500 секунд позволило расширить пространство поиска и сформировать более сложный ансамбль, однако не привело к росту точности классификации. Это показывает, что для данного набора данных оптимальное решение было найдено уже при меньшем временном ограничении, а дальнейшее увеличение вычислительного бюджета не даёт существенного выигрыша в качестве, влияя лишь на структуру и разнообразие итогового ансамбля моделей.

Затем время выполнения было снижено с 300с до 200с, результаты обучения представлены на рисунке 19.

```

=== METRICS ===
Overlap: 0
Target in X: False
Accuracy: 0.9998403321092129

Classification report:
      precision    recall  f1-score   support
0       0.999722    0.999722    0.999722    3584.00000
1       0.999888    0.999888    0.999888    8932.00000
accuracy       0.999840    0.999840    0.999840    0.99984
macro avg      0.999805    0.999805    0.999805    12526.00000
weighted avg   0.999840    0.999840    0.999840    12526.00000

=== SEARCH STATISTICS ===
auto-sklearn results:
Dataset name: 6a3fc08e-db7e-11f0-ab5-03d8126f6abb
Metric: accuracy
Best validation score: 0.999758
Number of target algorithm runs: 18
Number of successful target algorithm runs: 1
Number of crashed target algorithm runs: 9
Number of target algorithms that exceeded the time limit: 8
Number of target algorithms that exceeded the memory limit: 0

=== LEADERBOARD ===
      rank  ensemble_weight      type      cost  duration
model_id
5         1          1.0  gradient_boosting  0.00242  9.040874

=== MODELS INFO ===
[5: {'model_id': 5, 'rank': 1, 'cost': 0.002419110976715766, 'ensemble_weight': 1.0, 'data_preprocessor': <autosklearn.pipeline.components.data_preprocessing.DataPreprocessorChoice object at 0x7f37b4ed4df0>, 'balancing': Balancing(random_state=1), 'feature_preprocessor': <autosklearn.pipeline.components.feature_preprocessing.FeaturePreprocessorChoice object at 0x7f37b4ef2250>, 'classifier': <autosklearn.pipeline.components.classification.ClassifierChoice object at 0x7f37b4ef2730>, 'sklearn_classifier': HistGradientBoostingClassifier(early_stopping=True, l2_regularization=1.7108930238364161e-10, learning_rate=0.010227728124541558, max_iter=512, max_leaf_nodes=25, min_samples_leaf=4, n_iter_no_change=19, random_state=1, validation_fraction=0.1759114608225653, warm_start=True)}]

```

Рисунок 19. Обучение при ограничении в 200с.

Сокращение времени работы Auto-sklearn до 200 секунд привело к заметному упрощению процесса автоматического поиска моделей. В условиях жёсткого временного ограничения система не смогла сформировать полноценный ансамбль, о чём свидетельствуют предупреждения о невозможности построения ensemble и тот факт, что в итоговом решении использована только одна модель. Несмотря на это, достигнутая точность классификации осталась на очень высоком уровне (Accuracy ≈ 0.99984), лишь незначительно уступая результатам при большем вычислительном бюджете. Однако статистика поиска показывает резкое сокращение числа успешных запусков алгоритмов и увеличение доли неудачных и прерванных конфигураций, что указывает на ограниченное исследование пространства моделей. Таким образом, уменьшение времени обучения позволило сохранить высокую точность за счёт простоты задачи и информативности признаков, но привело к снижению устойчивости и разнообразия итогового решения, так как Auto-sklearn не получил возможности сформировать ансамбль из нескольких сильных моделей.

Выводы.

В ходе выполнения лабораторной работы были изучены принципы автоматизированного машинного обучения и получены практические навыки

использования библиотеки Auto-sklearn для решения задач регрессии и классификации. Эксперименты показали, что AutoML позволяет существенно упростить процесс подбора моделей и их гиперпараметров, автоматизируя все ключевые этапы — от предварительной обработки данных до формирования ансамблей и оценки качества.

При решении задачи прогнозирования задержек авиарейсов (dataset OpenML №42721) было установлено, что при заданных вычислительных ограничениях Auto-sklearn не смог найти регрессионные модели с удовлетворительным качеством. Низкие значения коэффициента детерминации R^2 и высокие значения ошибок MAE и RMSE свидетельствуют о сложности рассматриваемой задачи и недостаточной выразительной способности автоматически подобранных моделей в условиях ограниченного времени поиска. Увеличение числа вычислительных потоков не привело к улучшению качества прогнозирования, однако изменило характер поиска: возросла доля конфигураций, превысивших временные лимиты, и сократилось разнообразие итогового ансамбля. Это показывает, что распараллеливание не всегда обеспечивает рост качества и может сопровождаться снижением эффективности использования вычислительного бюджета.

В задаче классификации факта возгорания по данным датчиков (smoke_detection.csv) Auto-sklearn продемонстрировал высокую эффективность. Уже при ограничении времени в 300 секунд была достигнута идеальная точность классификации на тестовой выборке, что указывает на высокую информативность признаков и относительную простоту задачи. Увеличение времени поиска до 500 секунд не привело к росту точности, но позволило сформировать более разнообразный ансамбль моделей. При дальнейшем снижении времени до 200 секунд качество классификации осталось на высоком уровне, однако система не смогла сформировать ансамбль, и итоговое решение было представлено одной моделью. Это подчёркивает важность

вычислительного бюджета для устойчивости и надёжности AutoML-решений, особенно с точки зрения ансамблирования.

В целом результаты работы показывают, что Auto-sklearn является мощным инструментом для быстрого построения качественных моделей машинного обучения, особенно в задачах классификации. Однако эффективность AutoML существенно зависит от сложности данных и доступных вычислительных ресурсов. Для сложных регрессионных задач может потребоваться значительное увеличение времени поиска или предварительная инженерия признаков, тогда как для хорошо структурированных данных AutoML способен находить оптимальные решения даже при жёстких временных ограничениях.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

main.py

```
import time

import openml
import pandas as pd
import autosklearn.regression
import autosklearn.classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

def run_airdelays_task(
    dataset_id=42721,
    time_limit=1000,
    n_jobs=1,
    test_size=0.2,
    random_state=42
):
    # Загрузка данных
    dataset = openml.datasets.get_dataset(dataset_id)

    X, y, = dataset.get_data(target=dataset.default_target_attribute)
    print(X.shape)
    print("=== DESCRIPTION ===")
    print("Описание набора данных:\n", dataset.description)
    print("=== INFO ===")
    print(X.info())

    # Разделение на train/test
    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=test_size, random_state=random_state)

    # Обучение Auto-sklearn
    regressor = autosklearn.regression.AutoSklearnRegressor(
        time_left_for_this_task=time_limit,
        n_jobs=n_jobs,
    )

    start_time = time.time()
    regressor.fit(X_train, y_train)
    elapsed = time.time() - start_time
```

```

print(f"Время выполнения: {elapsed:.1f} сек")

# Оценка модели
y_pred = regressor.predict(X_test)

print("\n=== METRICS ===")
print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred, squared=False))
print("R^2:", r2_score(y_test, y_pred))

# Информация о поиске
print("\n=== SEARCH STATISTICS ===")
print(regressor.sprint_statistics())

print("\n=== LEADERBOARD ===")
print(regressor.leaderboard())

# Информация о лучших моделях
models_dict = regressor.show_models()
df_all = pd.DataFrame.from_dict(models_dict, orient='index')
columns_to_drop = ['data_preprocessor', 'feature_preprocessor',
'regressor']
df_clean = df_all.drop(columns=columns_to_drop, errors='ignore')
print("\n=== MODELS INFO ===")
print(df_clean)

def run_smoke_detection_task(
    file_path="smoke_detection_iot.csv",
    time_limit=200,
    n_jobs=1
):
    # Загрузка данных
    df = pd.read_csv(file_path)
    print("=== INFO ===")
    print(df.info())
    print("\n=== HEAD ===")
    print(df.head(10))
    print(f"\n=== STATISTIC ===")
    print(df.describe())

    # Удаляем предсказываемый класс из признаков
    X = df.iloc[:, :-1]
    # Получаем метки классов для каждого объекта
    y = df.iloc[:, -1]

    # Разделение на train/test
    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

    # Обучение Auto-sklearn
    cls = autosklearn.classification.AutoSklearnClassifier(
        time_left_for_this_task=time_limit,
        per_run_time_limit=time_limit//10,
        n_jobs=n_jobs
    )
    cls.fit(X_train, y_train)

```

```

# Оценка модели
y_pred = cls.predict(X_test)

print("\n=== METRICS ===")
# Проверка пересечения train / test
X_train_df = pd.DataFrame(X_train, columns=X.columns)
X_test_df = pd.DataFrame(X_test, columns=X.columns)
overlap = pd.merge(X_train_df, X_test_df, how='inner')
print("Overlap: ", len(overlap))
# Проверка утечки целевого признака
print("Target in X:", "Fire Alarm" in X.columns)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification report:")
report_dict = classification_report(y_test, y_pred, output_dict=True,
zero_division=0)
df_report = pd.DataFrame(report_dict).transpose()
print(df_report)

plt.figure(figsize=(6, 6))
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=cls.classes_,
)
disp.plot(cmap="Blues", values_format="d")
plt.title("Confusion Matrix (Smoke Detection)")
plt.tight_layout()
plt.savefig("ConfusionMatrix(SmokeDetection).png")
plt.close()
# Информация о поиске
print("\n=== SEARCH STATISTICS ===")
print(cls.sprint_statistics())

print("\n=== LEADERBOARD ===")
print(cls.leaderboard())

# Информация о лучших моделях
print("\n=== MODELS INFO ===")
print(cls.show_models())

if __name__ == "__main__":
    # run_airdelays_task()

    run_smoke_detection_task()

```